Firmware C: điều khiển hệ thống UART

Nios V thực thi chương trình • Avalon vận chuyển giao dịch

- Firmware là chương trình điều khiển hệ thống nhúng, được viết bằng C trong đề tài.
- Nhiệm vụ: nhận, xử lý và chuyển dữ liệu giữa JTAG terminal và UART vật lý.
- C quyết định gửi gì và khi nào; phần cứng UART tạo và nhận các bit trên dây.

GỢI Ý THUYẾT TRÌNH

Mở đầu bằng chức năng người dùng thấy: gõ dòng ở một terminal, Enter để chuyển sang terminal còn lại. Sau đó giới thiệu ba vai trò: firmware là chương trình, Nios V là CPU thực thi, Avalon là giao diện bus bên trong FPGA.

Nguồn: firmware/main.c; firmware/dti_uart.c; rtl/dti_uart.v

Firmware chạy trên CPU Nios V

CPU mềm và chương trình C là hai thành phần khác nhau

- Nios V/m được triển khai bằng logic trong FPGA; CPU lấy lệnh từ RAM on-chip.
- Code C được biên dịch thành lệnh RISC-V trong ELF rồi nạp để Nios V thực thi.
- CPU PC chạy công cụ build/nạp và terminal; firmware ứng dụng chạy trên Nios V.
- SOF cấu hình phần cứng FPGA; ELF cung cấp chương trình chạy trên CPU đó.

GỢI Ý THUYẾT TRÌNH

Nios V không phải một file C. Nó là bộ xử lý đã được tạo khi FPGA được cấu hình. RAM 128 KiB chứa chương trình, dữ liệu và stack. Có thể đổi ứng dụng chat sang scope bằng ELF khác khi phần cứng vẫn tương thích.

Nguồn: fpga/scripts/create_system.tcl; firmware/build.sh; fpga/program.sh

Firmware của đề tài làm những việc gì?

Ứng dụng chat hai chiều, điều khiển bằng thanh ghi

- Khởi tạo: kiểm tra System ID, chọn UART 8N1, bật LED ready.
- Nhận byte từ JTAG UART và UART riêng; lưu hai dòng nhập độc lập.
- Chờ Enter, chuẩn hóa CRLF rồi chuyển dòng sang phía đối diện.
- Nạp TX FIFO, đọc RX FIFO; theo dõi parity, tràn RX và timeout.

GỢI Ý THUYẾT TRÌNH

Firmware quyết định chính sách chat và xử lý lỗi. Baud 115200 được chọn trong RTL; C hiện chỉ cấu hình định dạng frame. Điều kiện Enter nằm trong chương trình C, không phải chức năng của FIFO hay bus Avalon.

Nguồn: firmware/main.c: uart_demo_init(), uart_demo_poll(), receive_char()

Avalon-MM là gì, dùng để làm gì?

Kết nối truy cập bộ nhớ và ngoại vi bên trong FPGA

- Avalon-MM là giao diện bus đọc/ghi theo địa chỉ; MM nghĩa là memory-mapped.
- Nios V phát yêu cầu; interconnect giải mã địa chỉ và định tuyến đến đích.
- RAM, JTAG UART, PIO và UART riêng cùng được CPU truy cập qua hệ thống này.
- Bridge chuyển Avalon-MM sang APB vì UART riêng của đề tài dùng giao diện APB.

GỢI Ý THUYẾT TRÌNH

Cách nói chính xác: firmware truy cập ngoại vi bằng MMIO; phần cứng Nios V phát giao dịch Avalon. Avalon không phải chương trình C và không phải đường USB nối với PC. IP JTAG UART nối Avalon trực tiếp, không qua bridge APB của UART riêng.

Nguồn: fpga/scripts/create_system.tcl; rtl/uart_avalon.v; rtl/avalon_apb_bridge.v

Quan hệ C → Nios V → Avalon → UART

Một lời gọi ghi thanh ghi được thực hiện qua các tầng sau



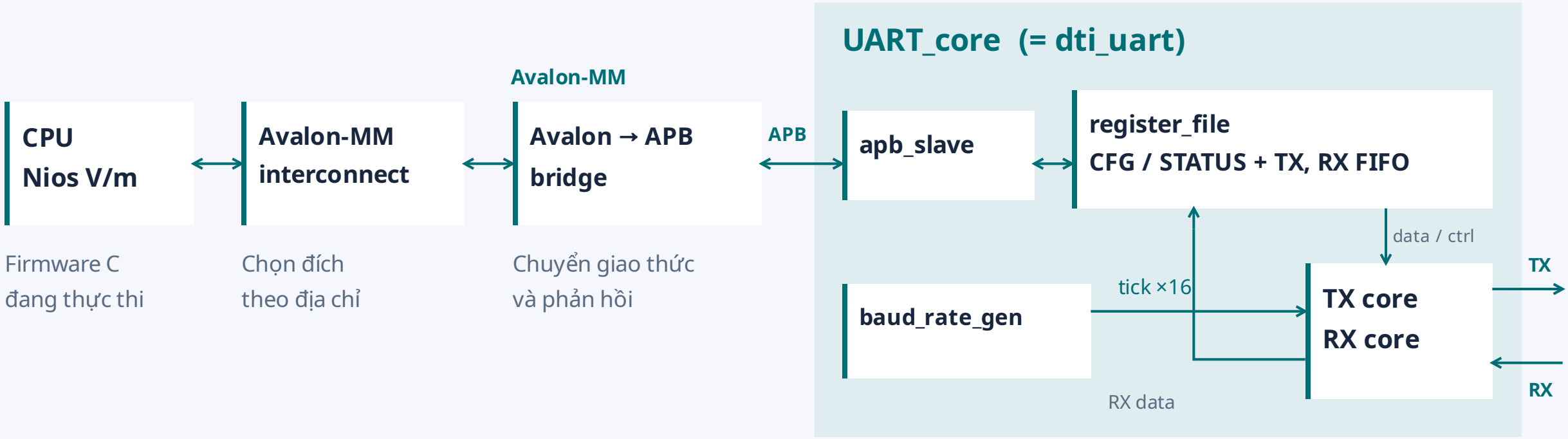
GỢI Ý THUYẾT TRÌNH

Ví dụ C muốn ghi byte h. Macro IOWR_32DIRECT thực hiện truy cập MMIO; CPU đưa yêu cầu ra bus. Bridge tạo các pha APB, slave tiếp nhận và register_file cập nhật thanh ghi. Mỗi khối có một nhiệm vụ, C không tự bật PSEL hoặc PENABLE.

Nguồn: firmware/uart_platform.h; rtl/avalon_apb_bridge.v; rtl/apb_slave.v; rtl/register_file.v

Sơ đồ Nios V → bridge → UART_core

Bridge ở ngoài UART_core; các khối cùng dùng clock hệ thống 50 MHz



Bus hai chiều: yêu cầu + phản hồi. Hình lược bỏ dây clock/reset, CTS/RTS và các dây trạng thái chi tiết.

GỢI Ý THUYẾT TRÌNH

UART_core là tên nhóm để trình bày, tương ứng module dti_uart. Trong RTL, uart_avalon chứa bridge và dti_uart. Register_file chứa hai FIFO; baud_rate_gen cấp tick cho TX/RX. Mũi tên bus hai chiều gồm yêu cầu và phản hồi, không phải hai giao dịch đồng thời.

Nguồn: rtl/uart_avalon.v; rtl/dti_uart.v; rtl/register_file.v; fpga/scripts/create_system.tcl

Vì sao cần Avalon–APB bridge?

Hai đầu dùng giao thức khác nhau, cần chuyển cách bắt tay giao dịch

- Phía Nios V: yêu cầu đọc/ghi Avalon-MM đi qua interconnect đến UART.
- Phía UART_core: APB slave nhận SETUP/ACCESS và trả PREADY, PRDATA, PSLVERR.
- Bridge chốt yêu cầu Avalon, tạo giao dịch APB rồi chuyển phản hồi về CPU.
- Bridge là RTL phần cứng; không xử lý Enter, chuỗi ký tự hay waveform UART.

GỢI Ý THUYẾT TRÌNH

Interconnect chọn ngoại vi theo địa chỉ; bridge chuyển giao thức; APB slave tiếp nhận giao dịch; register_file thực hiện tác động đọc/ghi. Nếu UART có giao diện Avalon trực tiếp thì không cần bridge APB này. Bản hiện tại giữ giao diện APB của UART theo thiết kế đề tài.

Nguồn: rtl/avalon_apb_bridge.v; rtl/apb_slave.v; rtl/uart_avalon.v

Bridge ánh xạ tín hiệu như thế nào?

Dữ liệu được chốt; tín hiệu điều khiển được sinh theo FSM

Phía Avalon-MM	Phía APB	Cách xử lý
av_address / av_writedata	PADDR / PWDATA	Chốt offset và dữ liệu ghi
av_write / av_byteenable	PWRITE / PSTRB	Chốt hướng và byte lane ghi
Yêu cầu được chấp nhận	PSEL / PENABLE	FSM sinh pha SETUP và ACCESS
av_readdata / av_response	PRDATA / PSLVERR	Trả dữ liệu đọc hoặc báo lỗi
waitrequest / response valid	PREADY	Chặn yêu cầu mới; trả phản hồi sau hoàn tất

GỢI Ý THUYẾT TRÌNH

Address ở cổng bridge là offset cục bộ 12 bit, không phải toàn bộ địa chỉ CPU. Khi đọc, bridge đặt PSTRB bằng 0. PREADY báo APB hoàn tất; nó không được nối thẳng vào waitrequest. Bridge nhận tối đa một yêu cầu đang xử lý, không hỗ trợ burst trong bản RTL này.

Nguồn: rtl/avalon_apb_bridge.v; fpga/ip/uart_avalon_hw.tcl

FSM bridge: một yêu cầu đi qua 4 trạng thái

IDLE → SETUP → ACCESS → RESPONSE → IDLE

- IDLE: sẵn sàng nhận Avalon; chốt địa chỉ, hướng, dữ liệu và byteenable.
- SETUP: PSEL = 1, PENABLE = 0; trình bày yêu cầu cho APB slave.
- ACCESS: PSEL = 1, PENABLE = 1; giữ yêu cầu đến khi PREADY = 1.
- RESPONSE: trả dữ liệu/trạng thái và bật response valid tương ứng đọc hoặc ghi.

GỢI Ý THUYẾT TRÌNH

Khi bridge khác IDLE, waitrequest chặn yêu cầu Avalon mới. ACCESS có thể kéo dài; APB slave của project có một wait cycle cố định và có thể chờ thêm. Chỉ commit thanh ghi khi giao dịch hoàn tất. Reset đưa bridge về IDLE và hệ thống cũng reset APB slave.

Nguồn: rtl/avalon_apb_bridge.v: state; rtl/apb_slave.v: pready, access_o

Ví dụ bridge khi CPU gửi hoặc nhận một byte

Hoàn tất giao dịch bus khác với hoàn tất frame UART

- Gửi h: ghi 0x68 tại 0x21000 → offset TX 0x00; sau đó ghi START tại 0x21008.
- Mỗi lần ghi là một giao dịch Avalon → APB riêng; START thành công mới enqueue.
- Nhận: kiểm tra RX_DONE tại 0x2100C, rồi đọc 0x21010 để pop RX FIFO.
- PRDATA quay về CPU qua bridge; nếu PSLVERR = 1 thì phản hồi Avalon báo lỗi.

GỢI Ý THUYẾT TRÌNH

Ghi START lúc FIFO đầy hoặc đọc RX lúc rỗng sẽ làm APB chờ, vì vậy driver kiểm tra trạng thái trước. Bridge không có timeout riêng; timeout C không thể giải cứu một truy cập bus đã bị giữ chờ. CPU chỉ kết luận TX phát hết khi đọc thấy TX_DONE, không dựa vào phản hồi ghi START.

Nguồn: firmware/dti_uart.c; rtl/register_file.v; rtl/avalon_apb_bridge.v

MMIO: chọn ngoại vi và thanh ghi bằng địa chỉ

Địa chỉ truy cập = địa chỉ nền + offset thanh ghi

Địa chỉ CPU	Thanh ghi / khối	Firmware sử dụng
0x20000	JTAG UART DATA	Nhận hoặc gửi byte console
0x21000	UART TX — offset 0x00	Ghi byte staging
0x21004 / 0x21008	CFG / CTRL	Chọn 8N1 / ghi START
0x2100C	STATUS	Kiểm tra TX_DONE, RX_DONE, parity
0x21010 / 0x21014	RX / FIFO_STATUS	Pop byte / kiểm tra đầy, rỗng, tràn

GỢI Ý THUYẾT TRÌNH

Ví dụ UART_BASE là 0x21000; cộng offset 0x0C được địa chỉ STATUS 0x2100C. Địa chỉ nền lấy từ system.h do BSP sinh. Ghi TX chỉ giữ staging; ghi START mới enqueue. Đọc RX có tác động pop nên driver kiểm tra RX_DONE trước.

Nguồn: firmware/dti_uart.h; firmware/uart_platform.h; fpga/scripts/create_system.tcl

Ví dụ code: gửi ký tự h vào TX FIFO

dti_try_queue() — kiểm tra chỗ trống, ghi byte, ghi START

- byte = 0x68 là ký tự h.
- TX_FULL = 1: trả DTI_AGAIN để thử lại sau.
- Ghi TX rồi START: đưa byte vào FIFO.
- DTI_OK là đã enqueue; TX_DONE mới báo toàn bộ TX đã phát xong.

TRÍCH CODE C

```
if (dti_fifo_status(dev) & DTI_TX_FULL)
    return DTI_AGAIN;
uart_mmio_write(dev->base + DTI_TX, byte);
uart_mmio_write(dev->base + DTI_CTRL, 1u);
return DTI_OK;
```

GỢI Ý THUYẾT TRÌNH

Thân hàm bên phải trích từ driver. Mỗi đọc/ghi MMIO tạo giao dịch bus; phần cứng TX tự phát frame sau khi lấy byte. Driver hiện dành cho một task sở hữu UART, tránh tác vụ khác chen vào giữa thao tác ghi TX và START.

Nguồn: firmware/dti_uart.c: dti_try_queue(); rtl/register_file.v

Firmware vận hành bằng vòng lặp polling

Khởi tạo một lần, sau đó liên tục phục vụ hai phía

- Xuất tối đa 16 byte JTAG khi IP còn chỗ; kiểm tra TX hoàn tất hoặc lỗi.
- Thử nhận 1 byte UART và 1 byte JTAG; xử lý ký tự, Enter và Backspace.
- Khi đợt TX trước đã xong: nạp tối đa 16 byte mới vào FIFO rồi quay lại vòng lặp.
- main.c xử lý ứng dụng; dti_uart.c và console.c truy cập hai ngoại vi.

GỢI Ý THUYẾT TRÌNH

Polling nghĩa là CPU chủ động kiểm tra trạng thái lặp lại; ứng dụng hiện không phục vụ dữ liệu bằng ngắt. Queue C nằm trong RAM, khác FIFO phần cứng 16 byte. Giới hạn công việc mỗi lượt giúp tránh dành quá lâu cho một phía; MMIO vẫn cần thời gian bus để hoàn tất.

Nguồn: firmware/main.c: uart_demo_poll(); firmware/console.c; firmware/dti_uart.c

Hai terminal: firmware chuyển dữ liệu ở giữa

Ví dụ JTAG → UART; chiều ngược lại thực hiện tương tự



GỢI Ý THUYẾT TRÌNH

Khi chưa Enter, firmware echo về bên nhập và giữ dòng. Khi Enter, nó chuyển cả dòng sang hàng đợi đầu ra. Chiều ngược: picocom → USB-UART → RX FIFO → C → JTAG UART → terminal 1. JTAG UART và UART vật lý là hai ngoại vi riêng; chip USB-UART trên ESP32 làm cầu USB–serial.

Nguồn: firmware/main.c: receive_char(); firmware/console.c; firmware/terminal_uart.sh

Ba vai trò cần nhớ khi trình bày

Chương trình quyết định • CPU thực thi • Bus kết nối

- Firmware C: chọn cấu hình, xử lý dòng, điều phối TX/RX và phản ứng với lỗi.
- CPU Nios V: thực thi lệnh máy, đọc/ghi RAM và thanh ghi ngoại vi.
- Avalon-MM: mang giao dịch CPU đến đúng đích; bridge đổi sang APB cho UART riêng.
- UART phần cứng: giữ FIFO và phát/nhận frame theo baud, độc lập với từng lệnh C.

GỢI Ý THUYẾT TRÌNH

Câu kết: “Firmware chạy trên Nios V, truy cập ngoại vi bằng MMIO qua Avalon; bridge chuyển sang APB; UART_core thực hiện truyền nhận.” Demo: gõ hello rồi Enter để chuyển dòng. UART_core trong hình tương ứng dti_uart. Bản 15 slide tổng hợp từ tài liệu trước và đối chiếu RTL bridge hiện tại.

Nguồn: Tổng hợp từ bản PDF chi tiết và source trong firmware/, rtl/, fpga/